

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

APPENDIX B

MANY-TO-MANY | SERVERPOD

 Copy as Markdown

Many-to-many

Many-to-many (n:m) relationships describe a scenario where multiple records from a table can relate to multiple records in another table. An example of this would be the relationship between students and courses, where a single student can enroll in multiple courses, and a single course can have multiple students.

Serverpod models these relationships with a separate model you define, called a junction table, that records the relation: neither table references the other directly, and the junction table holds the foreign keys of both.

When to use a junction table

Reach for a many-to-many relation when records on both sides can each relate to many records on the other, like students and courses. If a record on one side belongs to only one record on the other, a [one-to-many](#) relation is simpler.

A junction model also gives you a place to store data about the relationship itself. If the link between two records needs a date, a grade, or a status, those fields live on the junction model. A relationship that carries its own data always needs an explicit junction table, even when a plainer relation might otherwise do.

Defining the relationship

In the following examples we show how to configure an n:m relationship between `Student` and `Course`, with an `Enrollment` model as the junction table that stores the relationship explicitly.

Many tables

Both the `Course` and `Student` tables have a direct relationship with the `Enrollment` table but no direct relationship with each other.

```
# course.spy.yaml
class: Course
table: course
```



Ask AI

```
fields:
  name: String
  enrollments: List<Enrollment>?, relation(name=course_enrollments)
```

```
# student.spy.yaml
class: Student
table: student
fields:
  name: String
  enrollments: List<Enrollment>?, relation(name=student_enrollments)
```

Note that the `name` argument is different, `course_enrollments` and `student_enrollments`, for the many tables. This is because each row in the junction table holds a relation to each of the two related tables, `Course` and `Student`.

Junction table

The `Enrollment` table acts as the bridge between `Course` and `Student`. It contains foreign keys from both tables, representing the many-to-many relationship.

```
# enrollment.spy.yaml
class: Enrollment
table: enrollment
fields:
  student: Student?, relation(name=student_enrollments)
  course: Course?, relation(name=course_enrollments)
indexes:
  enrollment_index_idx:
    fields: studentId, courseId
    unique: true
```

The unique index on the combination of `studentId` and `courseId` ensures that a student can only be enrolled in a particular course once. If omitted a student would be allowed to be enrolled in the same course multiple times.

Managing junction records

To link a student to a course, create an `Enrollment` row with both foreign keys set and insert it:

```
var enrollment = Enrollment(
  studentId: student.id!,
  courseId: course.id!,
);
await Enrollment.db.insertRow(session, enrollment);
```

The `studentId` and `courseId` fields are the foreign keys Serverpod generated from the two relations. To remove the link, delete the junction row:

```
await Enrollment.db.deleteRow(session, enrollment);
```

Because `Student` and `Course` each hold a one-to-many relation to `Enrollment`, the generated `attach` and `detach` methods also operate on those relations. See [relation queries](#) for the attach and detach API.

Common errors

A few mistakes come up often when setting up a junction table:

- **Referencing the generated foreign-key fields.** Defining `student: Student?, relation(...)` generates an implicit `studentId` column on the junction table. Use these generated `<field>Id` names in indexes and filters, as the unique index above does. There is no separate field for you to declare.
- **Relation names must match on both sides.** The `name` argument pairs the two halves of a relation, so `course_enrollments` on `Course.enrollments` has to match `course_enrollments` on `Enrollment.course`. A mismatch is reported as a missing named relation when you run code generation.
- **Keep the relation fields persisted.** The relation fields on the junction are database columns. Marking one `!persist` drops the column, so the foreign key it backs can no longer be stored or queried.